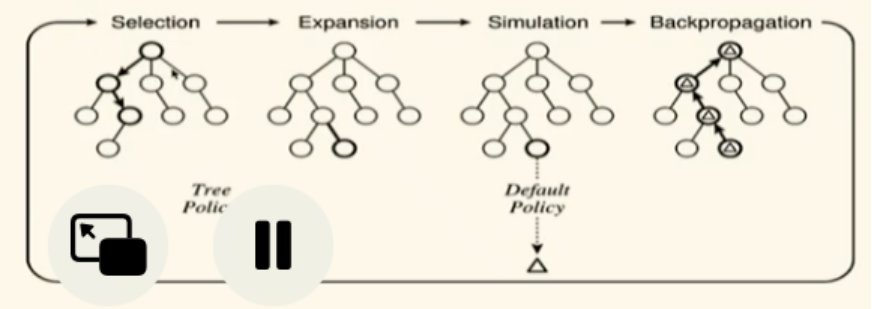
- Si formula sulla formula di bayes
- Non è sempre possibile saper 100% determinare ciò che succederà quando facciamo un azione
    - Dobbiamo prendere il più probabile, calcola che siamo in ambienti, e sensori con rumori
    - E qui abbiamo la regola di markov, che gioca con le seguenti assunzioni, markov decision process (MDP)
        - Il mondo è statico (non cambia mentre facciamo qualcosa)
            - Il mondo dipende dal presente **e non dal passato**
        - c'è un errore indipendente da ciò che facciamo
        - Il modello non fa errori di approssimazione
        - Il problema è che la regola di markov fa l'integrale per i calcoli, che è dispendioso
        - **non è deterministic**
        - Quindi, dato stesso stato, fare un azione non ci da uno stato sicuro
            - Noi dobbiamo fare l'azione migliore statisticamente parlando
        - MDP definito matematicametne:
            - S set di stati $s \in S$
            - A set di azioni $a \in A$
            - T modello transizionale $T(s, a, s')$
                - La probabilità che da stato s con azione a raggiungiamo stato s'
            - R funzione di reward $R(s, a, s')$
                - Prima avevamo una funzione di costo da minimizzare, ora abbiamo una funzione di reward da massimizzare
            - I stato inizia $i \in A$
            - G stato finale $g \in A$
            - Funzioni di utilità
        - Questo è un problema di reinforcement learning
            - Com'è che facciamo sapere se un azione ha veramente portato una soluzione oppure è stata sfortuna?
            - E questo è un problema dell'assegnamento del problema
            - Noi diamo un reward all'agente per dirli "bravo stai andando bene"
            - Il nostro obiettivo è massimizzare un reward comulativo nel tempo più grande possibile
                - Problema: siccome è una sommatoria, il problema potrebbe divergere o convergere

- Quindi dobbiamo stare lontani dagli stati che interrompono il tutto, è quindi possibile che arriviamo a stati non ottimali
- E un altro problema, come diamo il reward? Meglio tutto ora o un pò alla volta
    - E' meglio un pò alla volta, quindi per questo che si usa un valore γ che moltiplica il reward a seconda del passo
- In più, noi dobbiamo predire come andrà, ilchè rende il tutto più complicato, e questa è la funzione Q

$$Q(s,a) = \sum_{i=1}^{N}(P(s'|s,a)R(s,a,s') + max_{a'}\,\gamma Q(s',a'))$$

    - La sommatoria siccome dobbiamo prendere il reward migliore
        - Questa formula è tutto su markov
        - Nel reinforcement learning, **non conosciamo** l'ambiente, sono input -> output
    - La P è la probabilità, se prendiamo azione a con stato s, di avere s'
    - R è il possibile reward di s a con s'
    - γ per via del se vogliamo prendere subito o dopo
    - Poi noi vogliamo prendere la funzione Q con massima predizione a'
        - notiamo che, questa è l'unica parte che ha controllo l'agente il max con Q
        - Siccome è la scelta migliore per il futuro
        - Questa Q la ricalcoliamo sempre nel passo precedente
          Ed abbiamo 2 modi per calcolare la Q
            - Ad ogni passo ricalcoliamo Q con i valori correnti
            - Ad ogni passo ricalcoliamo la politica (magari da altezza a distanza)
              Nota che, la formula Q, che non sto a scrivere, la si può anche esprimere nell'errore che abbiamo commesso
            - Aka, la Q di ora - Q di prima
                - Questo è comodo quando noi non conosciamo l'ambiente
                - Sappiamo solo gli input, e prediciamo
            - E il nostro obiettivo è quello di portare il nostro segnale di errore vicino a 0

        - Se abbiamo errore positivo, allora l'agente ha sotto stimato quanto poteva muoversi
        - Se abbiamo un errore negativo, allora l'agente ha sovra stimato di quanto poteva muoversi
- Importante: l'algoritmo converge, più continua meno cambia
- Ad ogni passo, per potere
- Un altro sistema per trovare la Q migliore è attraverso la ricerca di monte carlo
    - 

    - Prendiamo un nodo, a caso, lo esploriamo, calcoliamo il reward, e poi propaghiamo il reward tutto indietro
    - Però ci sono tante policy da considerare
        - Quanto scendiamo?
        - Quale scegliamo?
        - Quanto continua?